

8051 MICRO CONTROLLERS

MICRO CONTROLLERS VS MICROPROCESSORS

MICROPROCESSOR:

A CPU built into a single VLSI chip is called a microprocessor.

It is a general-purpose device and additional external circuitry is added to make it a microcomputer.

The microprocessor contains arithmetic and logic unit (ALU), Instruction decoder and control unit, Instruction register, Program counter (PC), clock circuit (internal or external), reset circuit (internal or external) and registers.

But the microprocessor has no on chip I/O Ports, Timers, Memory etc.

For example, Intel 8085 is an 8-bit microprocessor and Intel 8086/8088 a 16-bit microprocessor.

MICROCONTROLLER:

A microcontroller is a highly integrated single chip, which consists of on chip CPU (Central Processing Unit), RAM (Random Access Memory), EPROM/PROM/ROM (Erasable Programmable Read Only Memory), I/O (input/output) – serial and parallel, timers, interrupt controller.

For example, Intel 8051 is 8-bit microcontroller and Intel 8096 is 16-bit microcontroller.

DISTINGUISH BETWEEN MICROPROCESSOR AND MICROCONTROLLER:

S. No	Microprocessor	Microcontroller
1	A microprocessor is a general purpose device which is called a CPU.	A microcontroller is a dedicated chip which is also called single chip computer.
2	A microprocessor does not contain on chip I/O Ports, Timers, Memories etc..	A microcontroller includes RAM, ROM, serial and parallel interface, timers, interrupt circuitry (in addition to CPU) in a single chip.
3	Microprocessors are most commonly used as the CPU in microcomputer systems.	Microcontrollers are used in small, minimum component designs performing control – oriented applications.

4	Microprocessor instructions are mainly nibble or byte addressable.	Microcontroller instructions are both bit addressable as well as byte addressable.
5	Microprocessor instruction sets are mainly intended for catering to large volumes of data.	Microcontrollers have instruction sets catering to the control of inputs and outputs.
6	Microprocessor based system design is complex and expensive.	Microcontroller based system design is rather simple and cost effective.
7	The Instruction set of microprocessors is complex with large number of instructions.	The instruction set of a Microcontroller is very simple with a smaller number of instructions. For, ex: PIC microcontrollers have only 35 instructions.
8	A microprocessor has zero status flag.	A microcontroller has no zero flag.

8051 MICROCONTROLLERS

8051 is one of the first and most popular microcontrollers also known as MCS-51. Intel introduced it in the year 1981. Initially, it came out as an N-type metal-oxide-semiconductor (NMOS) based microcontroller, but later versions were based on complementary metal-oxide-semiconductor (CMOS) technology. These microcontrollers were named 80C51, where C in the name tells that it is based on CMOS technology. It is an 8-bit microcontroller which means the data bus is 8-bit. Therefore, it can process 8 bits at a time. It is used in a wide variety of embedded systems like robotics, remote controls, the automotive industry, telecom applications, power tools, etc.

FEATURES OF 8051 MICROCONTROLLERS

The main features of the 8051-microcontroller architecture include the following.

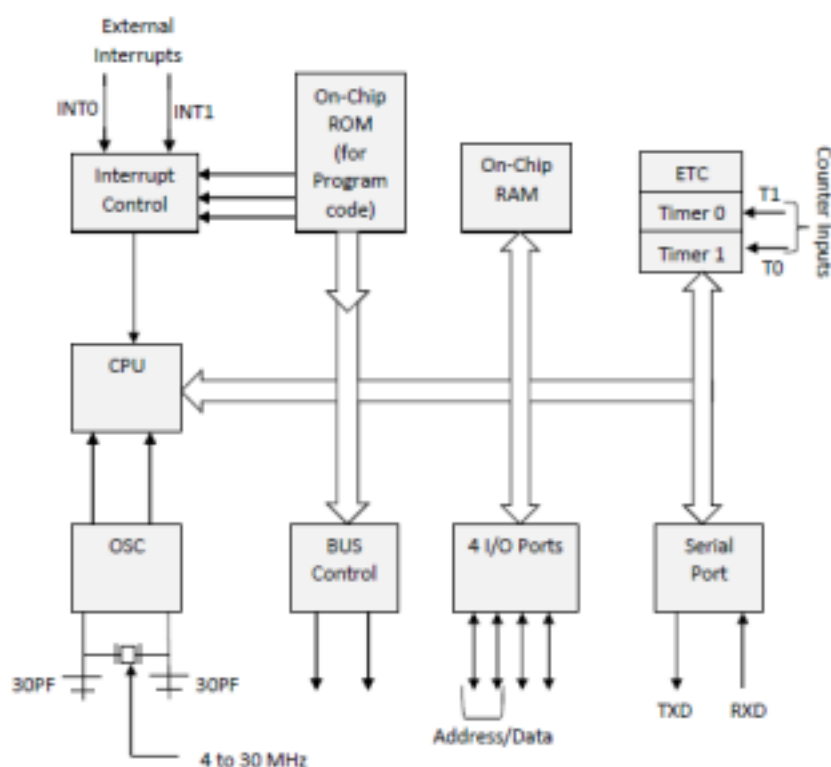
- 8-bit CPU through two Registers A & B.
- 8K Bytes – Internal ROM and it is a flash memory that supports while programming the system.
- 256 Bytes – Internal RAM where the first RAM with 128 Bytes from 00H to 7FH is once more separated into four banks through 8 registers in every bank, addressable registers -16 bit & general purpose registers – 80.
- The remaining 128 bytes of the RAM from 80H to FFH include Special Function Registers (SFRs). These registers control various peripherals such as Serial Port, Timers, all I/O Ports, etc.
- Interrupts like External-2 &

Internal-3

- Oscillator & CLK Circuit.
- Control Registers like PCON, SCON, TMOD, TCON, IE, and IP.
- 16-bit Timers or Counters -2 like T0 & T1.
- Program Counter – 16 bit & DPRT (Data Pointer).
- I/O Pins – 32 which are arranged like four ports such as P0, P1, P2 & P3.
- Stack Pointer (SP) – 8bit & PSW (Processor Status Word).
- Serial Data Tx & Rx for Full-Duplex Operation

BLOCK DIAGRAM OF 8051 MICROCONTROLLER

The Intel 8051 contains two separate buses for both program and data. So, it has two distinctive memory spaces of 64K x 8 size for both program and data. It is based on an 8-bit central processing unit with an 8-bit accumulator and another 8-bit B register as main processing blocks. Other portions of the architecture include few 8 bit and 16-bit registers and 8-bit memory locations. It has some amount of data RAM built in the device for internal processing. This area is used for stack operations and temporary storage of data. 8051 is supported with on-chip peripheral functions like I/O ports, Timers/ Counters, Serial communication port.



In the following diagram, the system bus connects all the support devices to the CPU. The system bus consists of an 8-bit data bus, a 16-bit address bus and bus control signals. All other devices like program memory, ports, data memory, serial interface, interrupt control, timers, and the CPU are all interfaced together through the system bus.

Internal RAM and ROM

ROM

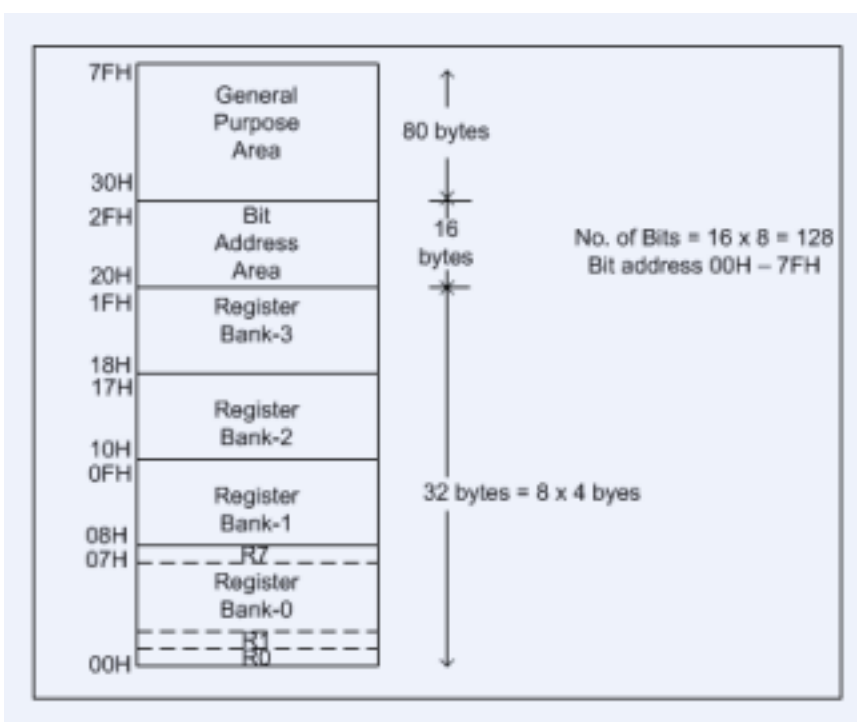
A code of 4K memory is incorporated as on-chip ROM in 8051. The 8051 ROM is a non-volatile memory

meaning that its contents cannot be altered and hence has a similar range of data and program memory, i.e, they can address program memory as well as a 64K separate block of data memory.

RAM

The 8051 microcontroller is composed of 128 bytes of internal RAM. This is a volatile memory since its contents will be lost if power is switched off. These 128 bytes of internal RAM are divided into 32 working registers which in turn constitute 4 register banks (Bank 0-Bank 3) with each bank consisting of 8 registers (R0 - R7).

There are 128 addressable bits in the internal RAM.



Four General Purpose Parallel Input/Output Ports

The 8051 microcontroller has four 8-bit input/output ports. These are:

PORT P0: When there is no external memory present, this port acts as a general-purpose input/output port. In the presence of external memory, it functions as a multiplexed address and data bus. It performs a dual role.

PORT P1: This port is used for various interfacing activities. This 8-bit port is a normal I/O port i.e. it does not perform dual functions.

PORT P2: Similar to PORT P0, this port can be used as a general-purpose port when there is no external memory but when external memory is present it works in conjunction with PORT P0 as an address bus. This is an 8-bit port and performs dual functions.

PORT P3: PORT P3 behaves as a dedicated I/O port

Interrupt Control

An event which is used to suspend or halt the normal program execution for a temporary period of time in order to serve the request of another program or hardware device is called an interrupt. An interrupt can either be an internal or external event which suspends the microcontroller for a while and thereby obstructs the sequential flow of a program.

There are two ways of giving interrupts to a microcontroller – one is by sending software instructions and the other is by sending hardware signals. The interrupt mechanism keeps the normal program execution in a "put on hold" mode and executes a subroutine program and after the subroutine is executed, it gets back to its normal program execution. This subroutine program is also called an interrupt handler. A subroutine is executed when a certain event occurs.

In 8051, 5 sources of interrupts are provided. They are:

- a) 2 external interrupt sources connected through INT0 and INT1
- b) 3 external interrupt sources- serial port interrupt, Timer Flag 0 and Timer Flag 1. The pins connected are as follows:

- 1. ALE (Address Latch Enable) - Latches the address signals on Port P0
- 2. EA (External Address) - Holds the 4K bytes of program memory
- 3. PSEN (Program Store Enable) - Reads external program memory
- 4. RST (Reset) - Reset the ports and internal registers upon start up

Serial Data Communication

A method of establishing communication among computers is by transmitting and receiving data bits is a serial connection network. In 8051, the SBUF (Serial Port Data Buffer) register holds the data; the SCON (Serial Control) register manages the data communication and the PCON (Power Control) register manages the data transfer rates. Further, two pins - RXD and TXD, establish the serial network.

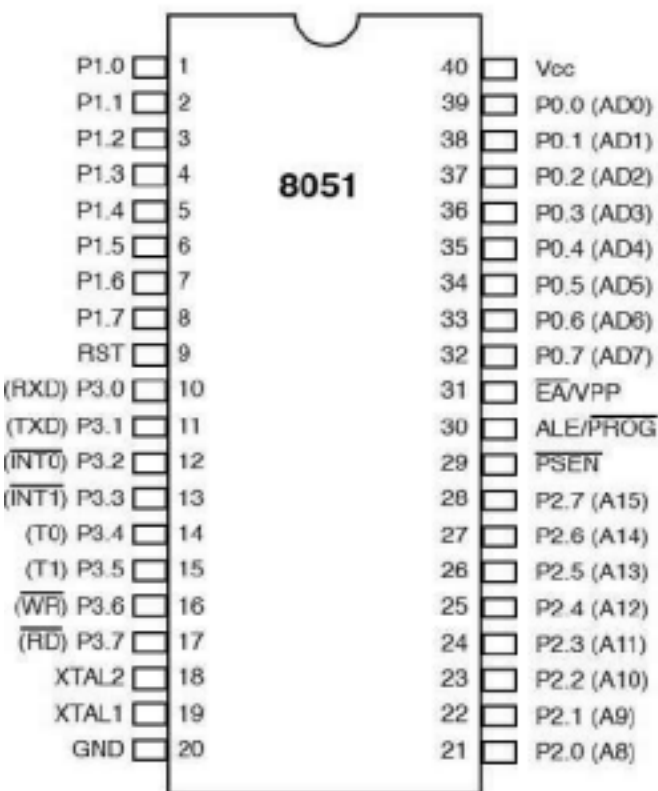
The SBUF register has 2 parts – one for storing the data to be transmitted and another for receiving data from outer sources. The first function is done using TXD pin and the second function is done using RXD pin.

There are 4 programmable modes in serial data communication. They are:

- 1. Serial Data mode 0 (shift register mode)
- 2. Serial Data mode 1 (standard UART)
- 3. Serial Data mode 2 (multiprocessor mode)
- 4. Serial Data mode 3

PIN DIAGRAM OF 8051

The pin diagram of 8051 microcontroller consists of 40 pins as shown below. A total of 32 pins are set away into four Ports such as P0, P1, P2 and P3. Where, each port contains 8 pins.



Port1 (Pin1 to Pin8): Port1 includes pin1.0 to pin1.7 and these pins can be configured as input or output pins.

Pin 9 (RST): Reset pin is used to Reset 8051 Microcontroller by giving a positive pulse to this Pin.

Port3 (Pin 10 to 17): The Port3 Pins are similar to port1 pins and can be used as universal Input or output pins. These pins dual-function Pins and the function of each Pin is given as:

Pin 10 (RXD): RXD pin is a Serial Asynchronous Communication Input or Serial synchronous Communication Output.

Pin 11 (TXD): Serial Asynchronous Communication Output or Serial Synchronous Communication Clock Output.

Pin 12 (INT0): Input of Interrupt 0

Pin 13 (INT1): Input of Interrupt 1

Pin 14 (T0): Input of Counter 0 clock

Pin 15 (T1): Input of Counter 1 clock

Pin 16 (WR): Writing Signal to write content on external RAM.

Pin 17 (RD): Reading Signal to read contents of external RAM.

Pin 18 and 19 (XTAL2, XTAL1): X2 and X1 pins are input output pins for the oscillator. These pins are used to connect an internal oscillator to the microcontroller.

Pin 20 (GND): Pin 20 is a ground pin.

Port2 (Pin 21 to Pin28): Port2 includes pin21 to pin28 which can be configured as Input Output Pins. But this is only possible when we don't use any external memory. If we use external memory, then these pins will work as high order address bus (A8 to A15).

Pin 29 (PSEN): This pin is used to enable external program memory. If we use an external ROM for storing the program, then logic 0 appears on it, which indicates Micro controller to read data from the memory.

Pin 30 (ALE): Address Latch Enable pin is an active high-output signal. If we use multiple memory chips, then this pin is used to distinguish between them. This Pin also gives program pulse input during programming of EPROM.

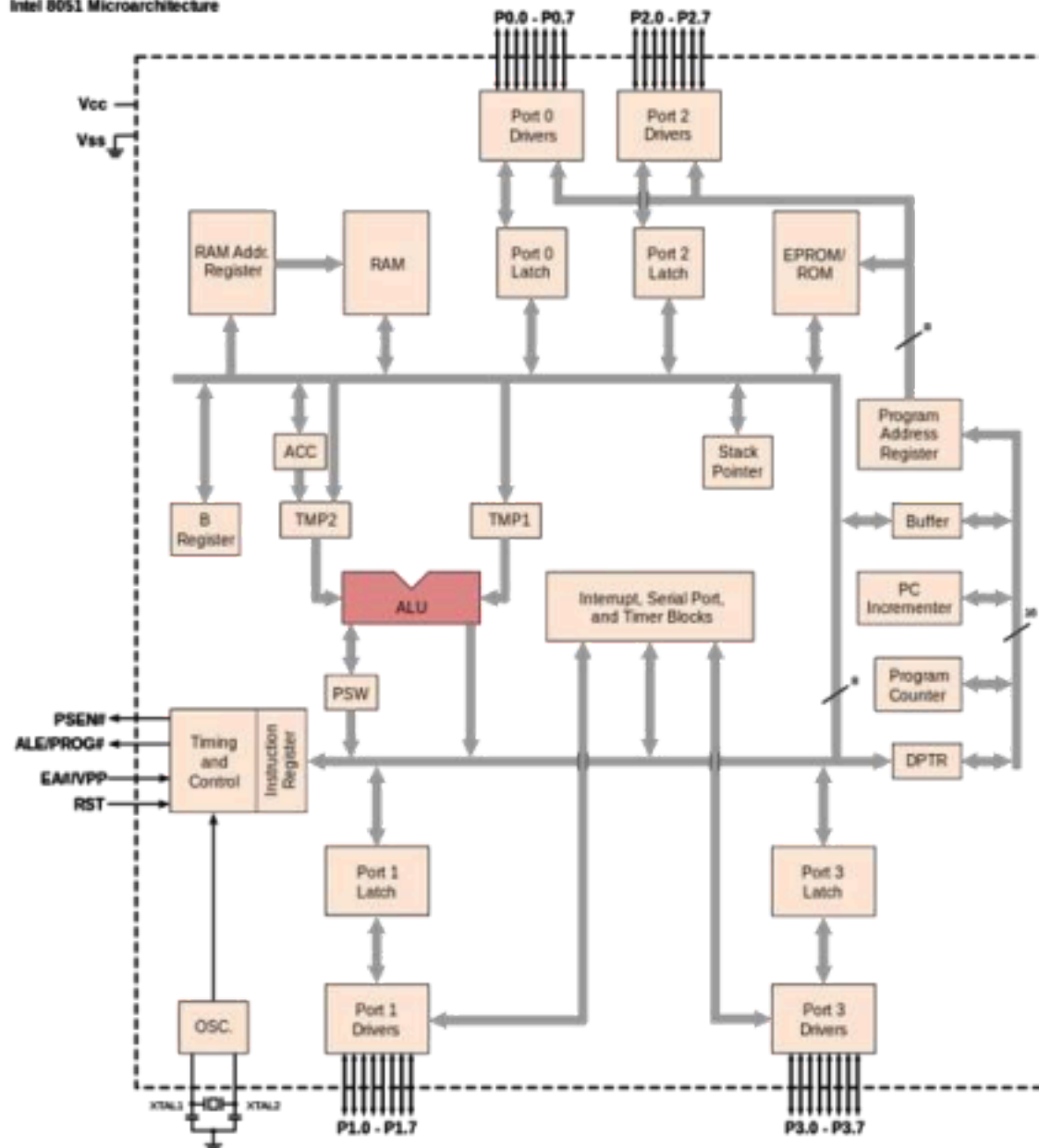
Pin 31 (EA): If we have to use multiple memories then the application of logic 1 to this pin instructs the Microcontroller to read data from both memories: first internal and then external.

Port 0 (Pin 32 to 39): Similar to the port 2 and 3 pins, these pins can be used as input output pins when we don't use any external memory. When ALE or Pin 30 is at 1, then this port is used as data bus: when the ALE pin is at 0, then this port is used as a lower order address bus (A0 to A7)

Pin40 (VCC): This VCC pin is used for power supply.

ARCHITECTURE OF 8051

Functional organization of the microcontroller 8051 consists of following units,



CENTRAL PROCESSING UNIT

The CPU is the brain of the microcontrollers expected task reading user's programs and executing the as per instructions stored there in. Its primary elements are an Accumulator (AC), Stack Pointer (SP) Program Counter (PC), Program Status Word (PSW), Data Pointer (DTPR) and few more 8-bit register.

ARITHMETIC LOGIC UNIT (ALU)

The arithmetic / logic unit performs the computing functions; it includes the accumulator, temporary register, arithmetic and logic circuits. The temporary register is used to hold data during an arithmetic / logic operation.

The result is stored in the accumulator register.

ACCUMULATOR

The accumulator register (ACC or A) act as an operand register, in case of some instructions. This may either

be implicit or specified in the instruction. □ The ACC register has been allotted on address in the on-chip special function □ register bank.

PROGRAM STATUS WORD (PSW)

This set of flags contains the status information and is considered as one of the special function registers.

D7	D6	D5	D4	D3	D2	D1	D0
PSW.7	PSW.6	PSW.5	PSW.4	PSW.3	PSW.2	PSW.1	PSW.0
CY	AC	FO	RS1	RS0	OV	---	P

Figure 3: Program Status Word (PSW)

PSW.1: User Definable Flag

PSW.2: OV: Overflow Flag

PSW.3: RS0: Register Bank Select Bit 0

PSW.4: RS1: Register Bank Select Bit 1

PSW.5: FO: Flag 0 available for general purpose

PSW.6: AC: Auxiliary Carry Flag

PSW.7: CY: Carry Flag

The bits PSW.3 and PSW.4 are denoted as RS0 and RS1. These bits are used to select the bank register of the RAM location.

STACK POINTER (SP)

This 8-bit register is incremented before the data is stored onto the stack using PUSH or CALL instructions. This register contains 8-bit stack top address. The stack may be defined anywhere in the on-chip 128-byte RAM. After reset, the SP register is initialized to 07. After each write two stack operation, the 8-bit contents of the operand are stored onto the stack, after incrementing the SP register by 1.

Thus, if SP contains 07H, the forthcoming PUSH operation will store the data at address 08H in the internal RAM. The SP content will be incremented to

08. The 8051 stack is not a top-down data structure, like other Intel processors. This register has also been allotted on address in the special function register bank.

DATA POINTER (DTPR)

This 16-bit register contains a higher byte (DPH) and the lower byte (DPL) of a 16-bit external data RAM address. It is accessed as a 16-bit register or two 8-bit registers as specified above. It has been allotted to two address in the special function register bank, for its two bytes DPH and DPL.

PORT 0 TO 3 LATCHES AND DRIVES

This four latches and driver pairs are allotted to each of the four on-chip I/O ports. These latches have been allotted address in the special function register bank. Using the allotted addresses, the users can communicate with these ports. These are identified as P0, P1 and P3.

SERIAL DATA BUFFER

The serial data buffer internally contains two independent registers. One of them is a transmit buffer which is necessarily a parallel-in serial-out (PISO) register. The other is called receive buffer which is a serial-in parallel-out (SIPO) register. Loading a byte to the transmit buffer initiates serial transmission of that byte. The serial data buffer is identified as SBUF and is one of the special function registers.

If a byte is written to SBUF, it initiates serial transmission and if the SBUF is read, it ☐ reads received serial data.

TIMER REGISTER

These two 16-bit register can be accessed as the lower and upper bytes. For examples, TL0 represents the lower byte of the timing register 0, while TH0 represents the upper byte of the timing register 0. Similarly, TL1 and TH1 represent lower and higher byte of the timing register 1. All these registers can be accessed using the four addresses allotted to them which lie in the special function register (SFR) address range, i.e. 80H to FF.

CONTROL REGISTER

The special function register IP, IE, TMOD, TCON, SCON and PCON contain control and status information for interrupts, Timer / Counters and serial port. All of the registers have been allotted addresses in the special function register bank of 8051.

TIMING AND CONTROL UNIT

This unit derives all the necessary timing and control signals recovered for the internal operation of the circuit. It also derives control signals recovered for controlling the system bus.

OSCILLATOR

This circuit generates the basic timing clock signal for the operation of the circuit using crystal oscillator

INSTRUCTION REGISTER

This register decodes the Opcode of an instruction to be executed and gives information to the timing and control unit to generate necessary signal for the execution of the instruction.

EPROM and Program Address Register

This block provides an on-chip EPROM and a mechanism to internally address it. (Note that EPROM is not available in all versions of 8051)

RAM and RAM Address Register

These blocks provide internal 128 bytes of RAM and a mechanism to address it

internally. **SFR (Special Function Register) Register Bank**

This is a set of special function registers, which can be addressed using their respective addresses which lie in the range 80H to FFH . Finally, the interrupt, serial port and timer units control and perform their specific function under the control of the timing and control unit.

ADDRESSING MODES OF 8051

There are five ways of addressing operands.

1. Register addressing
2. Direct addressing
3. Register- Indirect addressing
4. Immediate addressing
5. Indexed addressing

REGISTER ADDRESSING MODE

If the operand is specified in register it is called register addressing mode. It permits access to eight registers (R0- R7) of the register bank. There are four banks of eight registers. one of the four-register bank is selected by a 2 bit field in the program status word (PSW). Other registers are A, B, AB and DPTR.

For

example,

MOV A, Rn

$(A) \leftarrow (Rn)$

Move the contents of Rn into Accumulator.

DIRECT ADDRESSING MODE

In direct addressing mode, the address of the operand is specified in the instruction. Direct addressing has operands as byte or bit. Direct addressing of byte provides operation on one of the following.

1. Lower 128 bytes of internal data RAM.
2. Special Function Registers.

Direct bit addressing provides operation on the following.

1. RAM locations 00 - 1FH are assigned to the register banks and stack.
2. RAM locations 20 - 2FH are assigned for bit-addressable space .
3. RAM locations 30 - 7FH are available for byte-sized data.

Although the entire 128 bytes of RAM can be accessed using direct addressing mode, it is most often used to access RAM locations 30 - 7FH. This is due to the fact that register bank locations are accessed by the register names of R0- R7, but there is no such name for other RAM locations.

MOV R0,40H ;move content ofRAM location 40H in R0

RAM locations 0 to 7 are allocated to bank 0 registers R0- R7. These registers can be accessed in two ways.

MOV A, 04 ; is same as

MOV A, R4 ; copyR4 into A

REGISTER INDIRECT ADDRESSING MODE

If the address of the operand is specified in register it is called register indirect addressing mode.

The registers R0 and R1 are used for this purpose .ie, R2- R7 cannot be used to hold the address of an operand located in RAM. When R0 and R1 are used to hold the address of the RAM location, they must be preceded by @ sign.

Example

MOV A, @R0 ; move contents ofRAM whose addressis held by R0 into A

MOV @R1, B ; move contents of B into RAM location whose address is held
byR1

IMMEDIATE ADDRESSING MODES

In this addressing mode, the operand on which the operation has to be performed is specified in the

MOV A, #25H ; load 25H into A

MOV DPTR,#2550H is same as,

MOV DPL, #50H

MOV DPH, #25H

INDEXED ADDRESSING MODE AND ON-CHIP ROM ACCESS

This is an indirect instruction used to access the program memory. in this instruction the operand on which the operation is to be performed is not specified directly. The data pointer or PC register may act as the base register and register A act as index register. Thus,

the base register (DPTR or PC) and index register A are added to get the memory location.

For example,

MOVC A,@A+DPTR

$(A) \leftarrow ((A) + (DPTR))$

INSTRUCTION SET OF 8051

8051 Microcontroller have set of instruction to perform different operations. There are five group of instruction which are listed below.

- Arithmetic Instructions
- Logic Instructions
- Data Transfer Instructions
- Branch Instructions
- Bit-oriented Instructions

1.ARITHMETIC INSTRUCTION:

Arithmetic instructions perform several basic operations such as addition, subtraction, division, multiplication etc. After execution, the result is stored in the first operand.



1. ADD A,Rn;

Adds the register Rn to the accumulator

Description:

Instruction adds the register Rn (R0-R7) to the accumulator. After addition, the result is stored in the accumulator

Before execution:

A=2Eh R4=12h

After execution:

A=40h R4=12h

2. ADD A,@Ri

Adds the indirect RAM to the accumulator.

Ri: Register R0 or R1

Description:

Instruction adds the indirect RAM to the accumulator. Address of indirect RAM is stored in the Ri register (R0 or R1). After addition, the result is stored in the

accumulator. **Register address:**

R0=4Fh

Before execution:

A= 16h SUM= 33h

After execution:

A= 49h

3. ADDA, #DATA

Data: constant within 0-255 (0-FFh)

Description:

Instruction adds data (0-255) to the accumulator. After addition, the result is stored in the accumulator.

ADD A,#33h

Before execution:

A= 16h

After execution:

A= 49h)

4. SUBB A,direct

Subtracts the direct byte from the accumulator with a borrow

Direct: arbitrary register with address 0-255(0-FFh)

Description:

Instruction subtracts the direct byte from the accumulator with a borrow. If the higher bit is subtracted from the lower bit then the carry flag is set. As it is direct addressing, the direct byte can be any SFRs or general-purpose register with address 0-7Fh. (0-127 dec.). The result is stored in the accumulator.

SUBB A,Rx

Before execution:

A=C9h, DIF=53h, C=0

After execution:

A=76h, C=0

5. INC A - Increments the accumulator by 1

Description:

This instruction increments the value in the accumulator by 1. If the accumulator includes the number 255, the result of the operation will be 0.

Before execution:

A=E4h

After execution:

A=E5h

6. DEC A - Decrements the accumulator by 1

Description: Instruction decrements the value in the accumulator by 1. If there is a 0 in the accumulator, the result of the operation is FFh. (255 dec.)

Syntax: DEC A;

Byte: 1 (instruction code);

STATUS register flags:

No flags are affected;

Before execution:

A=E4h

After execution:

A=E3h

7. DIV AB - Divides the accumulator by the register B**Description:**

Instruction divides the value in the accumulator by the value in the B register. After division the integer part of result is stored in the accumulator while the register contains the remainder. In case of dividing by 1, the flag OV is set and the result of division is unpredictable. The 8-bit quotient is stored in the accumulator and the 8-bit remainder is stored in the B register.

Before execution:

A=FBh (251dec.) B=12h (18 dec.)

After execution:

A=0Dh (13dec.) B=11h (17dec.)

$13 \cdot 18 + 17 = 251$

8. DA A - Decimal adjust accumulator

Description: Instruction adjusts the contents of the accumulator to correspond to a BCD number after two BCD numbers have been added by the ADD and ADDC instructions.

The result in form of two 4-digit BCD numbers is stored in the accumulator. **Before execution:**

A=56h (01010110) 56BCD

B=67h (01100111)67BCD

After execution:

A=BDh (10111101)

After BCD conversion:

A=23h (00100011), C=1 (Overflow)

$(C+23=123) = 56+67$

9. MUL AB - Multiplies A and B

Description: Instruction multiplies the value in the accumulator with the value in the B register. The low-order byte of the 16-bit result is stored in the accumulator, while the high byte remains in the B register. If the result is larger than 255, the overflow flag is set. The carry flag is not affected.

Before execution:

A=80 (50h) B=160 (A0h)

After execution:

A=0 B=32h $A \cdot B = 80 \cdot 160 = 12800$ (3200h)

2.LOGICAL INSTRUCTION OF 8051 MICRO-CONTROLLER Logic

instructions perform logic operations upon corresponding bits of two registers.

After execution, the result is stored in the first operand.





ANL A,Rn - AND register to the accumulator A: accumulator Rn: any R register(R0-R7)

Instruction performs logic AND operation between the accumulator and Rn register.
The result is stored in the accumulator.

Syntax:

ANL A,Rn

Before execution:

A= C3h (11000011 Bin.) R5= 55h (01010101 Bin.)

After execution:

A= 41h (01000001 Bin.)

ORL A,Rn - OR register to the accumulator Rn: any R register (R0-R7) Instruction

performs logic OR operation between the accumulator and Rn register. The result is stored in the accumulator.

Syntax:

ORLA,Rn

Before execution:

A= C3h (11000011 Bin.) R5= 55h (01010101 Bin.)

After execution:

A= D7h (11010111 Bin.)

XRL A,Rn - Exclusive OR register to accumulator Rn: any R register (R0-R7)

Instruction performs exclusive OR operation between the accumulator and the Rn register. The result is stored in the accumulator.

Syntax:

XRL A,Rn

Before execution:

A= C3h (11000011 Bin.) R3= 55h (01010101 Bin.)

After execution:

A= 96h (10010110 Bin.)

CLR A - Clears the accumulator A: accumulator

Instruction clears the accumulator.

Syntax:

CLR A

After execution:

A=0

CPL A - Complements the accumulator

Instruction complements all the bits in the accumulator (1==>0, 0==>1).

Syntax:

CPL A

Before execution:

A=(00110110)

After execution:

A=(11001001)

RL A - Rotates the accumulator one-bit left A: accumulator

Eight bits in the accumulator are rotated one bit left, so that the bit 7 is rotated into the bit 0 position.

Syntax:

RL A

Before execution:

A= C2h (11000010 Bin.)

After execution:

A=85h (10000101 Bin.)

RR A - Rotates the accumulator one bit right

A: accumulator All eight bits in the accumulator are rotated one bit right so that the bit 0 is rotated into the bit 7 position.

Syntax:

RR A

Before execution:

A= C2h (11000010Bin.)

After execution:

A= 61h (01100001Bin.)

RLC A - Rotates the accumulator one bit left through the carry flag A: accumulator All eight bits in the accumulator and carry flag are rotated one bit left. After this operation, the bit 7 is rotated into the carry flag position and the carry flag is rotated into the bit 0 position.

Syntax:

RLC A

Before execution:

A= C2h(11000010 Bin.) C=0

After execution:

A= 85h(10000100Bin.) C=1

RRC A - Rotates the accumulator one bit right through the carry flag A: accumulator
All eight bits in the accumulator and carry flag are rotated one bit right. After this operation, the carry flag is rotated into the bit 7 position and the bit 0 is rotated into the carry flag position.

Syntax:

RRC A

Before execution:

A= C2h(11000010 Bin.) C=0

After execution:

A= 61h(01100001 Bin.) C=0

SWAP A - Swaps nibbles within the accumulator

A: accumulator

A nibble refers to a group of 4 bits within one register (bit0-bit3 and bit4-bit7). This instruction interchanges high and low nibbles of the accumulator.

Syntax:

SWAPA

Before execution:

A=E1h (11100001)bin.

After execution:

A=1Eh (00011110)bin.

3. DATA TRANSFER INSTRUCTION OF 8051

These
copy

instructions are used to
the content of source
operand to Destination
operand





MOV A,Rn - Moves the Rn register to the accumulator

The instruction moves the Rn register to the accumulator. The Rn register is not affected. Syntax

MOV A,Rn

Before execution:

R3=58h

After execution:

R3=58h A=58h

MOV A,@Ri - Moves the indirect RAM to the accumulator

Instruction moves the indirectly addressed register of RAM to the accumulator. The register address is stored in the Ri register (R0 or R1). The result is stored in the accumulator. The register is not affected.

Syntax:

MOV A,@Ri

Register Address SUM=F2h R0=F2h

Before execution:

SUM=58h

After execution:

A=58h SUM=58h

MOV A,#data - Moves the immediate data to the accumulator

Instruction moves the immediate data to the accumulator.

Syntax:

MOV A, #28

After execution:

A=28h

MOV direct,@Ri - Moves the indirect RAM to the direct byte

Instruction moves the indirectly addressed register of RAM to the direct byte.

The register is not affected.

Syntax:

MOV Rx,@Ri

Register Address SUM=F3

Before execution:

SUM=58h R1=F3

After execution:

SUM=58h TEMP=58h

MOVC A,@A+DPTR - Moves the code byte relative to the DPTR to the accumulator

Instruction first adds the 16-bit DPTR register to the accumulator. The result of addition is then used as a memory address from which the 8-bit data is moved to the accumulator.

4. BIT-ORIENTED INSTRUCTIONS

Similar to logic instructions, bit-oriented instructions perform logic operations. The difference is that these are performed upon single bits.



ANL C,bit - AND direct bit to the carry flag C: Carry flag

Bit: any bit of RAM,

Instruction performs logic AND operation between the direct bit and the carry flag.

Syntax:

ANL C,bit

Before execution:

ACC= 43h (01000011 Bin.) C=1

After execution:

ACC= 43h(01000011 Bin.) C=0

CLR C - clears the carry flag

C: Carry flag, Instruction clears the carry flag. After execution: C=0

CLR bit - clears the direct bit

Bit: any bit of RAM, Instruction clears the specified bit.

Syntax:

CLR P0.3

Before execution:

P0.3=1 (input pin)

After execution:

P0.3=0 (output pin)

CPL bit - Complements the direct bit

Bit: any bit of RAM, Instruction complements the specified bit of RAM (0==>1, 1==>0). Syntax:

CPL P0.3

Before execution:

P0.3=1 (input pin)

After execution:

P0.3=0 (output pin)

CPL C - Complements the carry flag

C: Carry flag, Instruction complements the carry flag (0==>1, 1==>0).

Syntax:

CPL C

Before execution:

C=1

After execution:

C=0

MOV bit,C - Moves the carry flag to the direct bit C: Carry flag, Bit: any bit of RAM

Instruction moves the carry flag to the direct bit. After executing the instruction, the carry flag is not affected.

Syntax:

MOVP1.2,C

After execution:

If C=0 P1.2=0 If C=1 P1.2=1

MOV C,bit - Moves the direct bit to the carry flag C: Carry flag, Bit: any bit of RAM

Instruction moves the direct bit to the carry flag. After executing the instruction, the bit is not affected.

Syntax:

MOV C, P1.4

After execution:

If P1.4=0 C=0 If P1.4=1 C=1

SETB C - Sets the carry flag

C: Carry flag, Instruction sets the carry flag.

Syntax:

SETB C

After execution: C=1

SETB bit - Sets the direct bit

Bit: any bit of RAM

Instruction sets the specified bit. The register containing that bit must belong to the group of the so-called bit addressable registers.

Syntax:

SETB P0.1

Before execution:

P0.1 = 34h (00110100) pin 1 is configured as an output

After execution:

P0.1 = 35h (00110101) pin 1 is configured as an input

5. BRANCH INSTRUCTION OF 8051 CONTROLLER

There are two kinds of branch instructions:

Unconditional jump instructions:

upon their execution a jump to a new location from where the program continues execution is executed.

Conditional jump instructions:

a jump to a new program location is executed only if a specified condition is met. Otherwise, the program normally proceeds with the next instruction.

**INTERRUPTS AND THEIR HANDLING IN 8051**

A single microcontroller can serve several devices. There are two ways to do that: • Interrupts

• Polling.

Interrupts

In the interrupt method, whenever the device needs its service, the device notifies the microcontroller by sending an interrupt signal. Upon receiving an interrupt signal, the microcontroller interrupts whatever it is doing and serves the device. The program associated with the interrupt is called the interrupt service routine (ISR) or interrupt handler.

Interrupt service routine

For every interrupt, there must be an interrupt service routine (ISR), or interrupt handler.

When an interrupt arises, the microcontroller runs the interrupt service routine. **Interrupt vector table**

The group of memory locations set aside to hold the addresses of ISRs is called the interrupt vector table.

Advantages of interrupts

The advantage of interrupts is that the microcontroller can serve many devices. Each device can get the attention of the microcontroller based on the priority assigned to it. In the interrupt method the microcontroller can also ignore (mask) a device request for service.

Steps in executing an interrupt

When an interrupt is activated, the microcontroller goes through the following steps. 1. It finishes the instruction it is executing and saves the address of the next instruction (PC) on the stack.

2. It also saves the current status of all the interrupts internally (i.e., not on the stack). 3. It jumps to a fixed location in memory called the interrupt vector table and holds the address of the interrupt service routine.

4. The microcontroller gets the address of the ISR from the interrupt vector table and jumps to it. It starts to execute the interrupt service subroutine until it reaches the last instruction of the subroutine, which is RETI (return from interrupt).

5. Upon executing the RETI instruction, the microcontroller returns to the place where it was interrupted. First, it gets the program counter (PC) address from the stack by popping the top two bytes of the stack into the PC. Then it starts to execute from that address.

Interrupt vector table

The interrupts in 8051 are allocated as follows

Interrupt	ROM Location(Hex)
Reset	0000
External hardware interrupt 0(INT0)	0003
Timer 0 interrupt(TF0)	000B
External hardware interrupt 1(INT1)	0013
Timer 1 interrupt(TF1)	001B
Serial COM interrupt (RI and TI)	0023

Enabling and disabling an interrupt

Upon reset, all interrupts are disabled (masked) i.e., none will be responded to microcontroller. There is a register called IE (interrupt enable) that is responsible for enabling (unmasking) and disabling (masking) the interrupts.

IE (interrupt enable) Register

D7 D0

EA	X	X	ES	ET1	EX0	ET0	EX0
----	---	---	----	-----	-----	-----	-----

EA (enable all):

If EA=0, no interrupt will be acknowledged.

If EA=1, each interrupt source is individually enabled or disabled by setting the enable bit.

ES - Enables or disables Serial port interrupt:

If ES=0, serial port interrupt is disabled.

ET1 Enables or disables the Timer 1 overflow interrupt:

If ET1 = 0, Timer 1 overflow interrupt is disabled

EX1 Enables or disables the external interrupt 1

If EX1 =0, external interrupt 1 is disabled

ET0 Enables or disables Timer 0 overflow interrupt

If ET0 = 0, Timer 0 overflow interrupt is disabled

EX0 Enables or disables the external interrupt 0

If EX0 = 0, external interrupt 0 is disabled

External interrupts INTO and INT1

8051 has two external hardware interrupts such as

1. INTO

2. INT1.

External hardware interrupts can be activated by two methods

1. Level triggered

2. Edge triggered.

Level-triggered interrupt

In the level-triggered mode, INTO and INT1 pins are normally high and if a low-level signal is applied to them, it triggers the interrupt. Then the microcontroller stops whatever it is doing and jumps to the interrupt vector table to service the interrupt. This is called a level-triggered or level-activated interrupt. The low-level signal at the INT pin must be removed before the execution of the last instruction of the interrupt service routine, (RETI) otherwise, another interrupt will be generated.

The IT0 and IT1 flag bits that determine level- or edge-triggered mode of the hardware interrupts.

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
-----	-----	-----	-----	-----	-----	-----	-----

Fig: TCON Register

During RESET TCON.0 (IT0) and TCON.2 (IT1) are both 0's. i.e., hardware interrupts INT0 and INT1 are low level triggered.

Edge-triggered interrupts

To make INT0 and INT1 as edge-triggered interrupts, TCON bits must be programmed. By making the TCON.0 and TCON.2 bits high with instructions such as "SETB TCON. 0" and "SETB TCON. 2", the external hardware interrupts of INT0 and INT1 become edge-triggered.

For example, the instruction "SETB CON. 2" makes INT1 as edge-triggered interrupt, in which, a high-to-low signal is applied to pin P3.3 (INT1). **Timer Interrupts**

8051 has two interrupts. They are

- 1) Timer 0 interrupt (T0)
- 2) Timer 1 interrupt (T1)

Whenever the timer rolls over, TF is raised. The timer interrupt in Interrupt Enable (IE) register is enabled and the microcontroller is interrupted and jumps to the interrupt vector table to service the interrupt service routine (ISR) which is located at 000BH.

Serial Communication Interrupt

The serial communication interrupt in 8051 are Transmit Interrupt (TI) and Receive Interrupt (RI).

Transmit interrupt (TI) is raised when the last bit of the framed data i.e., stop bit is transferred Receive Interrupt (RI) is raised when the entire frame of data including stop bit is received.

Interrupt Priority in 8051

When the 8051 is powered up, the priorities are assigned

Interrupt	Priority
External interrupt 0(INT0)	Highest
Timer interrupt 0(TF0)	
External interrupt 1(INT1)	
Timer	
interrupt 1(TF1)	Lowest
Serial communication (RI&TI)	

Interrupt Priority (IP) Register

D7 D0

-	-	-	PS	PT1	PX1	PT0	PX0
---	---	---	----	-----	-----	-----	-----

PS - Serial Port Interrupt

Priority level PT1 - Timer 1

Interrupt Priority level

PX1- External

Interrupt 1 Priority level PT0 - Timer 0 Interrupt Priority level PX0

- External Interrupt 0 Priority level Priority bit=1 assigns high priority. Priority bit =0 assigns low priority

TIMER AND COUNTER PROGRAMMING

The 8051 has two timers / counters. They can be used either as timers to generate a time delay or as counters to count events happening outside the micro-controller. The two timers in 8051 are:

- Timer 0 and
- Timer 1.

Basic registers of the timer

Both Timer 0 and Timer 1 are 16 bit wide. Since 8051 has an 8-bit architecture, each 16-bit timer is accessed as two separate registers of low byte and high byte. **Timer 0 registers**

The 16-bit register of Timer 0 is accessed as low byte and high byte. The low byte register is called TL0 (Timer 0 low byte) and the high byte register is referred to as TH0 (Timer 0 high byte).

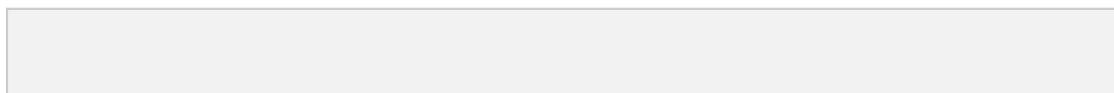


Fig. Timer 0 registers

Timer 1 registers

The 16-bit register of timer 1 is accessed as low byte and high byte. The low byte register is called TL1 (Timer 1 low) and the high byte register is called TH1 (Timer 1 high)



Fig. Timer 1 registers

TMOD (timer mode) register

TMOD is an 8 bit register in which the lower 4 bits are set for Timer 0 and the

higher 4 bits are set for Timer 1.

(MSB) (LSB)

GATE	C/ ☐	M1	M0	GATE	C/ ☐	M1	M0
-------------	------------------------------------	-----------	-----------	-------------	------------------------------------	-----------	-----------

Timer1 Timer 0

Fig: TMOD Register

GATE

When GATE = 0, the start and stop of timer are controlled by software by means of TR (timer start) bits TR0 and TR1. The timer is started by using the instruction “SETB TR0” for timer 0 and “SETB TR1” for timer 1 and stop by using the instruction “CLR TR1” for Timer 1, and “CLR TR0” for timer 0.

When GATE = 1, the start and stop of timer are controlled by hardware. C/ ☐

This bit in the TMOD register is used to decide whether the timer is used as a delay generator or an event counter. If C/ = 0, it is used as a timer. If C/ = 1, it is used as a counter.

M1, M0: (mode select bits)

M1	M0	Mode	Operating Mode
0	0	1	13-bit timer mode
0	1	1	16-bit timer mode
1	0	2	8-bit timer with auto reload
1	1	3	Split timer mode

Timer operating modes

Mode 0 (13-bit timer)

In this mode, TL0 and TH0 for timer 0 are used as 13-bit register, i.e., all the 8 bits of TL0 and 5 bits in TH0. The 13-bit counter can hold values between 0000 to 1FFFH in TH - TL. Therefore, when the timer reaches the maximum of 1FFFH, it rolls over to 0000, and TF is raised.



Fig: Mode 0

If $C/\square=0$, then the register is incremented after every machine cycle, i.e, at the rate of $1/12^{\text{th}}$ of the oscillator frequency. For the register to get incremented the control switch should be closed. This switch is logic controlled.

There are many ways in which the logic can be implemented.

- Case 1

TR0 = 1 (high)

Gate = 0 (low) and $\square = 0$ (low)

- Case 2

TR0 = 1 (high)

Gate = 0 (high) and $\square = 1$ (high)

Mode 0 programming

The following are the characteristics and operations of mode 0:

1. It is a 13-bit timer; therefore, it allows values of 0000 to 1FFFH to be loaded into the timer's registers TL and TH.
2. After TH and TL are loaded with a 13-bit initial value, the timer must be started. This is done by "SETB TR0" for Timer 0 and SETB TR1" for Timer 1.
3. After the timer is started, it starts to count up. It counts up until it reaches its limit of 1FFFH. When it rolls over from 1FFFH to 0000, it sets the timer flag to 1. When the timer flag is raised, one option to stop the timer is by using the instructions "CLR TR0" for timer 0 and "CLR TR1", Timer 1.
4. After the timer reaches its limit and rolls over, in order to repeat the process, the registers TH and TL must be reloaded with the original value, and TF must be reset

Mode 1 (16 bit timer)

In mode 1, 16 bit timer is used. The 16-bit counter can hold values between 0000 to FFFFH in TH - TL. Therefore, when the timer reaches its maximum of FFFFH, it rolls over to

0000, and TF is raised.

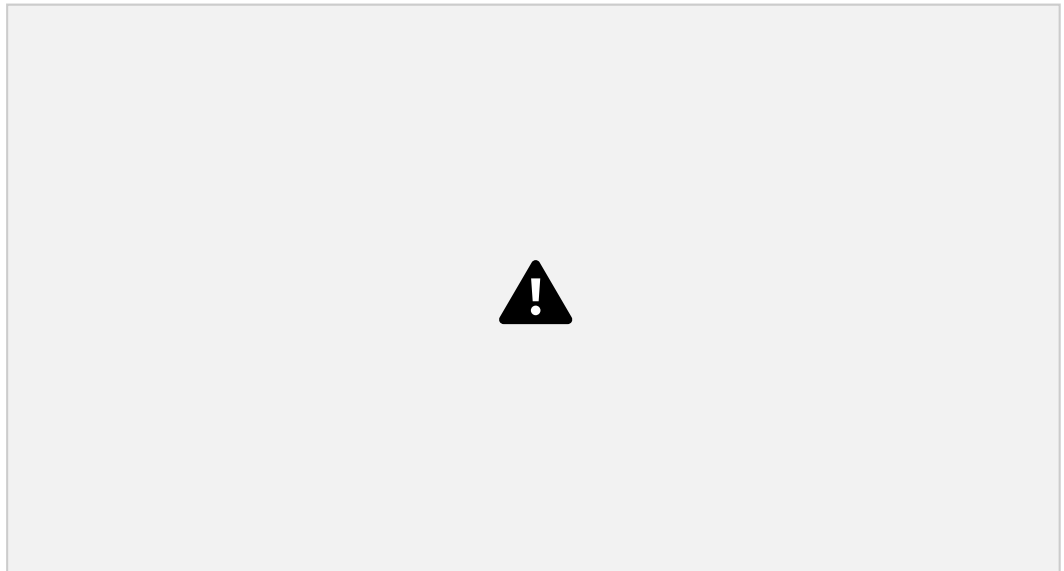


Fig: Mode 1

Mode 1 programming

The characteristics and operations of mode 1 are:

1. It is a 16-bit timer; therefore, it allows values of 0000 to FFFF_H to be loaded into the timer's registers TL and TH.
2. After TH and TL are loaded with a 16-bit initial value, the timer must be started. This is done by "SETB TR0" for Timer 0 and SETB TR1" for Timer 1.
3. After the timer is started, it starts to count up. It counts up until it reaches its limit of FFFF_H. When it rolls over from FFFF_H to 0000, it sets timer flag to 1. When the timer flag is raised, one option to stop the timer is using the instructions "CLR TR0" for timer 0 and "CLR TR1", for Timer 1.
4. After the timer reaches its limit and rolls over, in order to repeat the process, the registers. TH and TL must be reloaded with the original value, and TF must be reset to 0.

Mode 2 (8 – bit with auto reload)

In this mode, the timer register is 8 bits wide. TL0 for Timer 0 or TL1 for timer 1 is used for this purpose. This mode is also called the auto-reload mode as the timer generates an interrupt on overflow and after generating the interrupt, it reloads the preset value from TH0 into TL0. This preset value is put into TH0 through software.

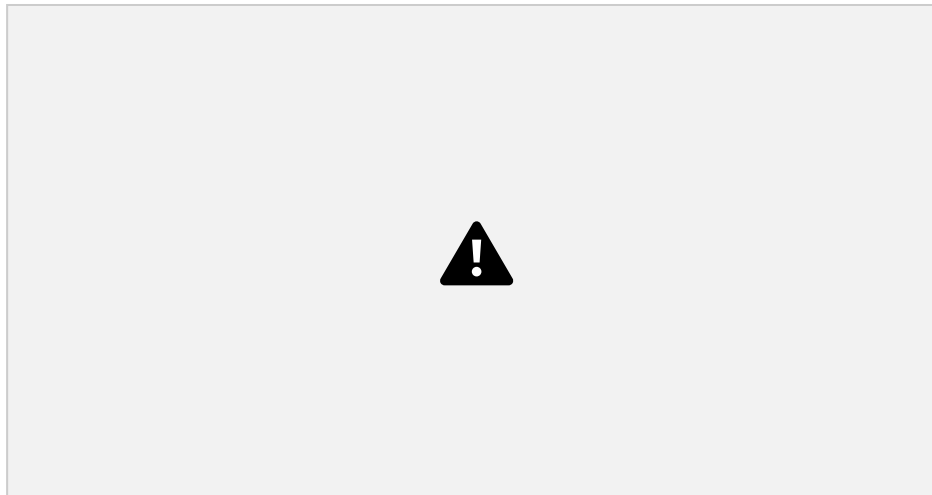


Fig: Mode 2- 8-bit timer with auto reloaded

Mode 2 programming

The following are the characteristics and operations of mode 2.

1. It is an 8-bit timer; therefore, it allows only values from 00 to FFH to be loaded into the timer's register TH.
2. After TH is loaded with the 8-bit value, the 8051 gives a copy to TL. And the timer is started. This is done by the instruction "SETB TR0" for Timer 0 and "SETB TR1, for Timer 1.
3. After the timer is started, it starts to count up by incrementing the TL registers. It counts up until it reaches its limit of FFH. When it rolls over from FFH to 00, it sets high the TF (timer flag). If Timer 0 is used TFO goes high; if Timer 1 is used TF1 is raised.
4. When the TL registers rolls from FFH to 00 and TF is set to 1, TL is reloaded automatically with the original value kept by the TH register.

To repeat the process, clear TF and without any programmer the original value is reload.

Mode 3 (Split timer mode)

If the Timer 0 is put into mode 3, then it acts as two 8-bit counters (TL0 and TH0 become two separate counters). All the timer 0 control bits (C/ , Gate, TR0, TF0 and ☐ ☐) are used by TL0 itself and TH0 register is locked into a timer function. TH0 is counting machine cycles and has taken over the use of TR1 and TF1 from Timer 1. Therefore, TH0 will now control Timer 1 interrupt. If the Timer 1 is put into mode 3, it just holds the count.

The effect is same as setting TR1 = 0, hence opening the switch.

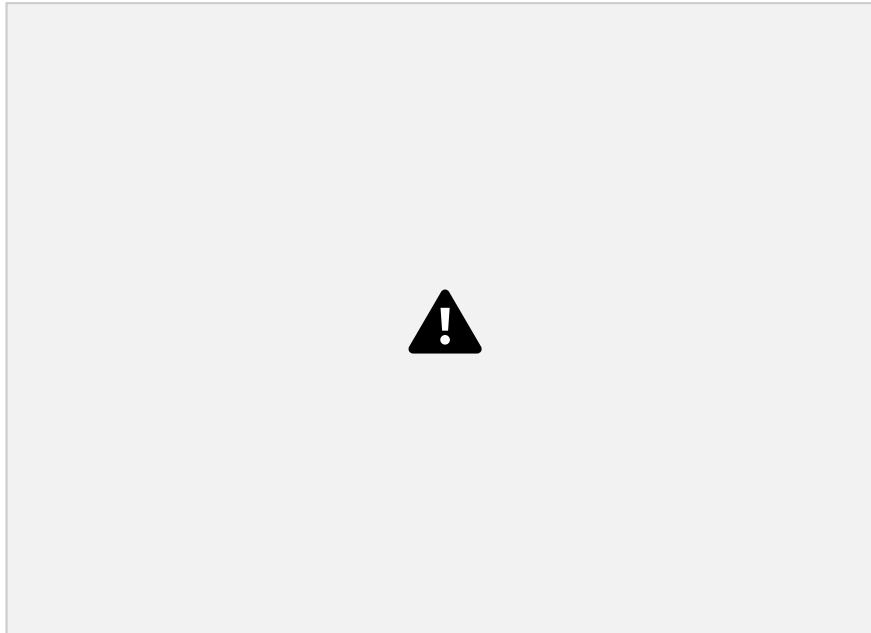


Fig : Mode 3

The start and stop of the timer is controlled by TR bit in timer control (TCON) Register.

TCON register

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
-----	-----	-----	-----	-----	-----	-----	-----

Fig: TCON Register

TF1: Timer 1 overflows flag

Set by hardware when the timer/counter overflows.

Cleared by hardware when the processor vectorsto the interrupt routine.

TR1: Timer 1 run control bit

Set/cleared by software to turn the timer/counter on/off.

TF0: Timer 0 overflow flag

Set by hardware when the timer/counter overflows.

Cleared by hardware when the processor vectorsto the interrupt routine.

TR0: Timer 0 run control bit

Set/cleared by software to turn the timer/counter on/off.

IE1: Interrupt 1 edge flag

Set by hardware when the external interrupt edge is detected. Cleared when the interrupt is processed.

IT1: Interrupt 1 type control bit

Set/ Cleared by software to specify the falling edge/low level triggered

external interrupts.



• When IT1 = 1 then ☐ •

When IT0 = 0 then ☐ falling edge triggered

IE0: Interrupt 0 edge flag is low-level triggered



Set by hardware when the external interrupt edge is

detected. Cleared when the interrupt is processed.

IT0: Interrupt 0 type control bit

Set/ Cleared by

software to specify the

falling edge/low level triggered external interrupts.

- When IT0 = 1 then ☐ is falling edge triggered
- When IT0 = 0 then ☐ is low-level triggered